Q1-> validate parenthesis by moving one bracket.

Code:

```
class Solution {
    public boolean canBeBalanced(String s) {
        int open = 0, close = 0;
        for (char c : s.toCharArray()) {
            if (c == '(') open++;
            else close++;
        }
        // unequal counts → impossible
        if (open != close) return false;

        if (isValid(s)) return true;

        // try one swap: check negative balance points
        int negativeCount = 0;
        int balance = 0;
        for (char c : s.toCharArray()) {
            if (c == '(') balance++;
            else balance--;
            if (balance < 0) negativeCount++;
        }

        // One swap can fix only one "dip"
        return negativeCount <= 1;
    }

    private boolean isValid(String s) {
        int balance = 0;
        for (char c : s.toCharArray()) {
            if (c == '(') balance++;
```

```
            else balance--;
            if (balance < 0) return false;
        }
        return balance == 0;
    }

    // quick test
    public static void main(String[] args) {
        Solution sol = new Solution();
        System.out.println(sol.canBeBalanced("()")); // true
        System.out.println(sol.canBeBalanced")("));  // true (swap)
        System.out.println(sol.canBeBalanced("())("));  // true (swap)
        System.out.println(sol.canBeBalanced("((")); // false
        System.out.println(sol.canBeBalanced")))("));  // false
    }
}
```

Q2-> https://learningmonkey.in/courses/placement-training-service-based/lessons/sequence-sum-in-python/

Code:

Given input three integers. The integers are taken in three variables, i,j,k.

Condition: always i and k should be less than j.

sequence sum is given as the sum of all the values from i to j and then from j to k.

Example: i=5, j =10, k =6

output: 5+6+7+8+9+10+9+8+7+6 = 75

Q3-> a[] ={“Hello”,”Bee”,”Tree”} , b[]={“World”,”By”,”Box”}

Return a list containing yes or no if the a[i] characters are present in b[i] -> yes else NO.

Code:

```
import java.util.*;


class Solution {
    public static List<String> checkStrings(String[] a, String[] b) {
        List<String> result = new ArrayList<>();


        for (int i = 0; i < a.length; i++) {
            boolean[] present = new boolean[256]; // ASCII
            for (char c : b[i].toCharArray()) {
                present[c] = true;
            }


            boolean ok = true;
            for (char c : a[i].toCharArray()) {
                if (!present[c]) {
                    ok = false;
                    break;
                }
            }
            result.add(ok ? "Yes" : "No");
        }
        return result;
    }


    public static void main(String[] args) {
        String[] a = {"Hello","Bee","Tree"};
        String[] b = {"World","By","Box"};
```

```
        System.out.println(checkStrings(a, b));
        // Output: [No, No, No]
    }
}
```

Q4-> No of friends = 4(N) , No of secs(time) = 10(T) . The friends are playing a game passing ball to each other in every second . and return who passed ball to whom in last sec .

Code:

```
 import java.util.*;


public class BallGame {
   public static String lastPass(int N, int T) {
      int position = 1;   // start with friend 1
      int dir = 1;        // 1 = forward, -1 = backward
      int from = -1, to = -1;

      for (int t = 1; t <= T; t++) {
         from = position;
         position += dir;  // move ball

         // check if at boundary → reverse
         if (position == N) dir = -1;
         else if (position == 1) dir = 1;

         to = position;
      }
      return from + " -> " + to;
   }

   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      int N = sc.nextInt(); // number of friends
      int T = sc.nextInt(); // number of seconds
      System.out.println(lastPass(N, T));
   }
}
```

Q5-> Remove Continuous substring to make a string unique(should not contain repeating characters)
input - abcbbck output – abck , remove substring – bbc

Code:

Q6 -> Count similar pair of substring (https://share.google/F7In2Xh8pzTUmpBQq)

Code:

```
class Solution {
    public int similarPairs(String[] words) {
        int count = 0;
        for(int i=0; i<words.length-1; i++){
            for(int j=i+1; j<words.length; j++){
                count += check(words[i], words[j]);
            }
        }
        return count;
    }
    private int check(String s, String t){
        Set sset = new HashSet();
        for(char c: s.toCharArray())    sset.add(c);
        Set tset = new HashSet();
        for(char c: t.toCharArray())    tset.add(c);
        return sset.equals(tset) ? 1 : 0;
    }
}
```

Q7-> Take the string of many words as 1 string input and check how many words are valid.

Validation conditions for a word:

1. Length of word must be greater than 2.
2. Must contain at least 1 vowel.
3. Must contain at least 1 consonant.
4. Word can only contain alphabets or alphanumeric (no special characters).

Task: Write a Java function to return the number of valid words.

Code:

```
public class ValidWordsChecker {


    public static int countValidWords(String input) {
        String[] words = input.split("\\s+"); // split by spaces
        int validCount = 0;


        for (String word : words) {
            if (isValid(word)) {
                validCount++;
            }
        }
        return validCount;
    }


    private static boolean isValid(String word) {
        if (word.length() <= 2) return false;


        boolean hasVowel = false, hasConsonant = false;


        for (char c : word.toCharArray()) {
            if (!Character.isLetterOrDigit(c)) {
                return false; // invalid character
            }
```

```
            if (Character.isLetter(c)) {
                char lower = Character.toLowerCase(c);
                if ("aeiou".indexOf(lower) >= 0) {
                    hasVowel = true;
                } else {
                    hasConsonant = true;
                }
            }
        }

        return hasVowel && hasConsonant;
    }

    public static void main(String[] args) {
        String input = "cat dog a1b2 xy z apple bcd123";
        System.out.println("Valid words: " + countValidWords(input));
    }
}
```

Q8-> Given a 0-indexed integer array **arr[]** and an integer **d**, return the count number of triplets (i, j, k) such that i < j < k and **(arr[i] + arr[j] + arr[k]) % d == 0**.

Code:

```
import java.util.HashMap;


public class Main {


  // Function to count the number of triplets (i, j, k)
  // such that i < j < k and (arr[i] + arr[j] + arr[k]) %
  // d == 0
  static int divisibleTripletCount(int[] arr, int d)
  {
    // Initialize variables
    int n = arr.length;
    int ans = 0;

    // Store the count of pairs with a pair sum modulo d
    HashMap<Integer, Integer> dupletSums
      = new HashMap<>();

    // Iterate over the array from the second element to
    // the last
    for (int k = 1; k < n; k++) {
      // Calculate the expected remainder for the
      // current element to form a divisible triplet
      int expected = (d - (arr[k] % d) + d) % d;

      // Add the count of pairs with the expected
      // remainder to the answer
      ans += dupletSums.getOrDefault(expected, 0);
```

```
        // Iterate over the previous elements
        for (int i = 0; i < k; i++) {
            // Calculate the sum of the current element
            // and the previous element modulo d
            int x = (arr[i] + arr[k]) % d;

            // Increment the count of pairs with the
            // calculated sum in the map
            dupletSums.put(
                x, dupletSums.getOrDefault(x, 0) + 1);
        }
    }

    // Return the total count of divisible triplets
    return ans;
}

public static void main(String[] args)
{
    // Initialize the input array
    int[] arr = { 1, 2, 3, 4, 5 };

    // Initialize the target divisor
    int d = 3;

    // Call the divisibleTripletCount function to count
    // the number of divisible triplets
    int count = divisibleTripletCount(arr, d);

    // Print the count of divisible triplets
    System.out.println(
```

```
        "The number of divisible triplets is: "
        + count);
    }
}
```

Q9-> Movie rating and duration question

https://leetcode.com/problems/maximum-subsequence-score/description/

Code:

```
public long maxScore(int[] speed, int[] efficiency, int k) {
    int n = speed.length;
    int[][] ess = new int[n][2];
    for (int i = 0; i < n; ++i)
        ess[i] = new int[] {efficiency[i], speed[i]};
    Arrays.sort(ess, (a, b) -> b[0] - a[0]);
    PriorityQueue<Integer> pq = new PriorityQueue<>(k, (a, b) -> a - b);
    long res = 0, sumS = 0;
    for (int[] es : ess) {
        pq.add(es[1]);
        sumS = (sumS + es[1]);
        if (pq.size() > k) sumS -= pq.poll();
        if (pq.size() == k) res = Math.max(res, (sumS * es[0]));
    }
    return res;
}
```

Q10-> PROBLEM SOLVING

I have been given two arrays - one contains the skill levels of employees, and the other contains the sizes of each team. I need to divide all employees into teams according to the given sizes.

The strength of a team is the sum of the minimum and maximum skill in that team. I have to form the teams in such a way that the total strength of all teams is as high as possible. Each employee must be placed in exactly one team. I have to return the maximum possible total team strength after dividing all employees as per the team sizes.

Sample Input:

empSkill = [1, 10, 5, 9, 9]

teamSize = [1, 1, 3]

Output: 48

Code:

```
import java.util.*;

public class MaxTeamStrength {
    public static int maximizeStrength(int[] empSkill, int[] teamSize) {
        Arrays.sort(empSkill);
        Arrays.sort(teamSize);

        int n = empSkill.length;
        int m = teamSize.length;

        // sort descending to take largest first
        int left = 0, right = n - 1;
        long total = 0;

        // first assign largest skills to all teams
```

```
        int[] maxVals = new int[m];
        for (int i = m - 1; i >= 0; i--) {
            maxVals[i] = empSkill[right--];
        }

        // now compute strengths
        for (int i = 0; i < m; i++) {
            if (teamSize[i] == 1) {
                // single-member team: min=max
                total += 2 * maxVals[i];
            } else {
                // multi-member team: min = smallest available
                total += maxVals[i] + empSkill[left];
                left += teamSize[i] - 1;
            }
        }

        return (int) total;
    }

    public static void main(String[] args) {
        int[] empSkill = {1, 10, 5, 9, 9};
        int[] teamSize = {1, 1, 3};
        System.out.println(maximizeStrength(empSkill, teamSize)); // Output: 48
    }
}
```

Q11-> **Problem Restated Correctly**

- Input: Binary string s of length n.
- Operation allowed: Swap **pairs** (i,i+1) with (j,j+1) where i and j are even indices.
- You can perform this operation **any number of times**.
- Goal: Find the **longest non-decreasing substring** (0s followed by 1s) possible **after any sequence of allowed swaps**.

Code:

```
import java.util.*;


public class MaxNonDecreasingBinaryPairs {


    // Compute longest non-decreasing substring
    public static int longestNonDecreasing(String s) {
        int maxLen = 1, curLen = 1;
        for (int i = 1; i < s.length(); i++) {
            if (s.charAt(i) >= s.charAt(i - 1)) {
                curLen++;
            } else {
                curLen = 1;
            }
            maxLen = Math.max(maxLen, curLen);
        }
        return maxLen;
    }


    // Maximize after pair swaps
    public static int maxLength(String s) {
        int n = s.length();
        List<String> pairs = new ArrayList<>();


        // Create pairs
```

```
        for (int i = 0; i < n; i += 2) {
            if (i + 1 < n) {
                pairs.add("" + s.charAt(i) + s.charAt(i + 1));
            } else {
                pairs.add("" + s.charAt(i)); // last single char if n is odd
            }
        }

        // Sort pairs by number of 0s descending
        pairs.sort((a, b) -> {
            int zerosA = (int) a.chars().filter(c -> c == '0').count();
            int zerosB = (int) b.chars().filter(c -> c == '0').count();
            return zerosB - zerosA;
        });

        // Merge pairs back to string
        StringBuilder sb = new StringBuilder();
        for (String p : pairs) sb.append(p);

        // Compute longest non-decreasing substring
        return longestNonDecreasing(sb.toString());
    }

    public static void main(String[] args) {
        String s = "1100101";
        System.out.println(maxLength(s)); // Output: maximum length
    }
}
```

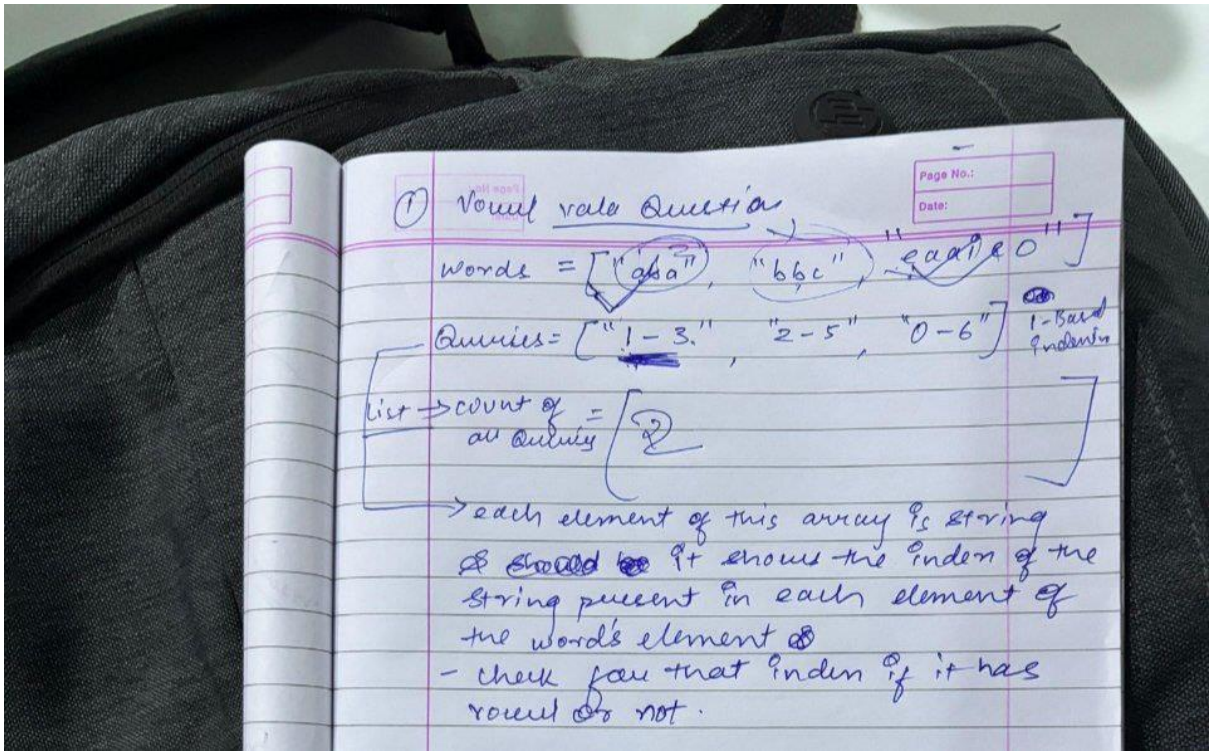
Q12-> perform k operation (Math.ceil wala question)

Code:

```
import java.util.*;


public class MaxAfterKOperations {


    public static int maxAfterKOperations(int[] nums, int k) {
        // Max-heap
        PriorityQueue<Integer> maxHeap = new PriorityQueue<>(Collections.reverseOrder());


        // Add all elements to the heap
        for (int num : nums) {
            maxHeap.add(num);
        }


        // Perform k operations
        for (int i = 0; i < k; i++) {
            int max = maxHeap.poll();   // Extract max
            int reduced = max / 2;      // Floor division
            maxHeap.add(reduced);       // Add back reduced value
        }


        // After k operations return the maximum element left
        return maxHeap.peek();
    }


    public static void main(String[] args) {
        int[] nums = {5, 19, 8, 1};
        int k = 3;


        int result = maxAfterKOperations(nums, k);
```

```
        System.out.println("Max element after " + k + " operations: " + result);
    }
}
```

Q13->



code:

```
import java.util.*;


public class VowelValueQuery {


    private static boolean isVowel(char c) {

        return "aeiou".indexOf(c) != -1;

    }


    public static int[] vowelQueries(String[] words, String[] queries) {

        int n = words.length;

        int[] valid = new int[n];


        // Step 1: Mark valid words (start and end with vowel)

        for (int i = 0; i < n; i++) {

            String w = words[i];

            if (isVowel(w.charAt(0)) && isVowel(w.charAt(w.length() - 1))) {

                valid[i] = 1;
```

```
            }
        }

        // Step 2: Prefix sum
        int[] prefix = new int[n+1];
        for (int i = 0; i < n; i++) {
            prefix[i+1] = prefix[i] + valid[i];
        }

        // Step 3: Answer queries
        int[] result = new int[queries.length];
        for (int i = 0; i < queries.length; i++) {
            String q = queries[i];
            String[] parts = q.split("-");
            int l = Integer.parseInt(parts[0]);
            int r = Integer.parseInt(parts[1]);
            result[i] = prefix[r+1] - prefix[l];
        }

        return result;
    }

    public static void main(String[] args) {
        String[] words = {"aba", "bbc", "eaeio"};
        String[] queries = {"0-2", "1-2", "0-2"};

        int[] ans = vowelQueries(words, queries);
        System.out.println(Arrays.toString(ans)); // [2,1,2]
    }
}
```

Q14->